LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Azhril Tirtha

2509106097

C1

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart

Gambar 1.1 Flowchart

Penjelasan singkat:

Flowchart dimulai dari **Start**, kemudian program melakukan **inisialisasi jumlah user dan jumlah data**. Setelah itu, program menampilkan **menu awal** yang berisi pilihan register, login, atau keluar.

Jika pengguna memilih **register**, maka pengguna memasukkan nama dan NIM, lalu data disimpan ke dalam array user dan kembali ke menu awal.

Jika memilih **login**, pengguna diminta memasukkan nama dan NIM. Program akan melakukan pengecekan dengan maksimal **3 kali percobaan**. Jika login gagal, percobaan akan bertambah. Jika berhasil, pengguna akan masuk ke **menu CRUD**. Jika gagal 3 kali, program akan berhenti.

Di dalam **menu CRUD**, pengguna dapat memilih:

- **Tambah data** → menambahkan data sayur jika kapasitas belum penuh
- **Lihat data** → menampilkan seluruh data jika tidak kosong
- **Ubah data** → memilih data berdasarkan nomor lalu mengubahnya
- **Hapus data** → menghapus data dan menggeser array
- **Keluar** → kembali ke menu awal

Semua proses dalam menu CRUD akan berulang sampai pengguna memilih keluar. Terakhir, jika pengguna memilih keluar dari menu awal, program akan menampilkan **Program selesai** dan flowchart berakhir di **End**.

2. Deskripsi Singkat Program

Program ini adalah aplikasi berbasis C++ untuk mengelola data toko sayur menggunakan konsep CRUD (Create, Read, Update, Delete). Program diawali dengan fitur register dan login, di mana pengguna harus memasukkan nama dan NIM dengan maksimal 3 kali percobaan. Setelah berhasil login, pengguna dapat menambahkan, melihat, mengubah, dan menghapus data sayur yang disimpan dalam array of struct. Program juga menggunakan fungsi dan prosedur untuk memisahkan tiap proses, serta berjalan secara berulang hingga pengguna memilih keluar.

3. Source Code

```
#ifndef VALIDASI_H
#define VALIDASI_H

#include <stdexcept>
using namespace std;

void cekAngka(int x) {
    if (x < 0) {
        throw runtime_error("Nilai tidak boleh negatif!");
    }
}

void cekIndex(int index, int max) {
    if (index < 0 || index >= max) {
        throw out_of_range("Index di luar batas!");
    }
}

#endif
```

```cpp
#include <iostream>
#include "validasi.h"
using namespace std;

#define MAX 100

struct Sayur {
    int id;
    string nama;
    int harga;
    int stok;
};

void tampilData(Sayur *data, int jumlah) {
    if (jumlah == 0) throw runtime_error("Data kosong!");

    for (int i = 0; i < jumlah; i++) {
        cout << i+1 << ". "
             << (data+i)->id << " | "
             << (data+i)->nama << " | "
             << (data+i)->harga << " | "
             << (data+i)->stok << endl;
    }
}

void tambahData(Sayur *data, int *jumlah) {
    if (*jumlah >= MAX) throw runtime_error("Data penuh!");

    cout << "ID: "; cin >> (data+*jumlah)->id;
    cout << "Nama: "; cin >> (data+*jumlah)->nama;

    cout << "Harga: "; cin >> (data+*jumlah)->harga;
    if (cin.fail()) throw invalid_argument("Harga harus angka!");
    cekAngka((data+*jumlah)->harga);

    cout << "Stok: "; cin >> (data+*jumlah)->stok;
    cekAngka((data+*jumlah)->stok);

    (*jumlah)++;
}
```

```
void hapusData(Sayur *data, int *jumlah) {
    if (*jumlah == 0) throw runtime_error("Data kosong!");

    int idx;
    cout << "Pilih nomor: "; cin >> idx;

    cekIndex(idx-1, *jumlah);

    for (int i = idx-1; i < *jumlah-1; i++) {
        *(data+i) = *(data+i+1);
    }
    (*jumlah)--;
}

void bubbleSortNama(Sayur *data, int n) {
    for (int i=0;i<n-1;i++)
        for (int j=0;j<n-i-1;j++)
            if ((data+j)->nama > (data+j+1)->nama)
                swap(*(data+j), *(data+j+1));
}

void selectionSortHarga(Sayur *data, int n) {
    for (int i=0;i<n-1;i++) {
        int max=i;
        for (int j=i+1;j<n;j++)
            if ((data+j)->harga > (data+max)->harga)
                max=j;
        swap(*(data+i), *(data+max));
    }
}

void insertionSortStok(Sayur *data, int n) {
    for (int i=1;i<n;i++) {
        Sayur key = *(data+i);
        int j=i-1;
        while (j>=0 && (data+j)->stok > key.stok) {
            *(data+j+1)=*(data+j);
            j--;
        }
        *(data+j+1)=key;
    }
```

```
}

int linearSearchNama(Sayur *data, int n, string cari) {
    for (int i=0;i<n;i++)
        if ((data+i)->nama == cari) return i;
    return -1;
}

void bubbleSortID(Sayur *data, int n) {
    for (int i=0;i<n-1;i++)
        for (int j=0;j<n-i-1;j++)
            if ((data+j)->id > (data+j+1)->id)
                swap(*(data+j), *(data+j+1));
}

int binarySearchID(Sayur *data, int n, int cari) {
    int low=0, high=n-1;
    while (low<=high) {
        int mid=(low+high)/2;
        if ((data+mid)->id == cari) return mid;
        else if ((data+mid)->id < cari) low=mid+1;
        else high=mid-1;
    }
    return -1;
}

int main() {

    Sayur data[MAX];
    int jumlah=0, menu;

    do {
        cout << "\n=== MENU ===\n";
        cout << "1. Tambah\n2. Tampil\n3. Hapus\n";
        cout << "4. Sort Nama (ASC)\n5. Sort Harga (DESC)\n6. Sort Stok
(ASC)\n";
        cout << "7. Cari ID (Binary)\n8. Cari Nama (Linear)\n9. Keluar\n";
        cout << "Pilih: "; cin >> menu;

        try {
            switch(menu) {
```

```
            case 1: tambahData(data,&jumlah); break;
            case 2: tampilData(data,jumlah); break;
            case 3: hapusData(data,&jumlah); break;

            case 4: bubbleSortNama(data,jumlah); break;
            case 5: selectionSortHarga(data,jumlah); break;
            case 6: insertionSortStok(data,jumlah); break;

            case 7: {
                int id; cout<<"Cari ID: "; cin>>id;
                bubbleSortID(data,jumlah);
                int hasil=binarySearchID(data,jumlah,id);
                if (hasil!=-1)
                    cout<<"Ditemukan: "<<(data+hasil)->nama<<endl;
                else cout<<"Tidak ditemukan\n";
                break;
            }

            case 8: {
                string nama; cout<<"Cari Nama: "; cin>>nama;
                int hasil=linearSearchNama(data,jumlah,nama);
                if (hasil!=-1)
                    cout<<"Ditemukan ID: "<<(data+hasil)->id<<endl;
                else cout<<"Tidak ditemukan\n";
                break;
            }
        }
    }
    catch (const invalid_argument& e) {
        cout<<"Error: "<<e.what()<<endl;
        cin.clear(); cin.ignore(1000,'\n');
    }
    catch (const out_of_range& e) {
        cout<<"Error: "<<e.what()<<endl;
    }
    catch (const runtime_error& e) {
        cout<<"Error: "<<e.what()<<endl;
    }

} while(menu!=9);
```

```
    return 0;
}
```

Gambar 3.1 Source Code

4. Hasil Output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 1
ID: 2
Nama: bayam
Harga: 13000
Stok: 2000
```

Gambar 4.1 Output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 2
1. 2 | bayam | 13000 | 2000
```

Gambar 4.2 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 3
Pilih nomor: 1

=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 3
Error: Data kosong!
```

Gambar 4.3 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 4
```

Gambar 4.4 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 5
```

Gambar 4.5 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 6
```

Gambar 4.6 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 7
Cari ID: 2
Ditemukan: bayam
```

Gambar 4.7 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 8
Cari Nama: bayam
Ditemukan ID: 2
```

Gambar 4.8 output

```
=== MENU ===
1. Tambah
2. Tampil
3. Hapus
4. Sort Nama (ASC)
5. Sort Harga (DESC)
6. Sort Stok (ASC)
7. Cari ID (Binary)
8. Cari Nama (Linear)
9. Keluar
Pilih: 9
PS D:\KULIAH INFOR AZHRIL\praktikum-apl\post-test\post-test-apl-7>
```

Gambar 4.9 output

5. Langkah-langkah GIT

5.1 GIT Init

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git init
Initialized empty Git repository in D:/praktikum-apl/post-test/post-test-apl-1/.git/
```

Gambar 5.1 Git init

Digunakan untuk membuat repository Git baru di dalam sebuah folder agar folder tersebut dapat dikelola dengan Git.

5.2 GIT Add

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git add .
```

Gambar 5.2 Git add

Digunakan untuk menambahkan file atau perubahan file ke dalam *staging area* sebelum disimpan ke repository.

5.3 GIT Commit

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git commit -m "first commit"
[main (root-commit) 7fca732] first commit
 2 files changed, 76 insertions(+)
 create mode 100644 2509106097-MUHAMMADAZHRILTIRTHA-PT-1.cpp
 create mode 100644 2509106097-MUHAMMADAZHRILTIRTHA-PT-1.exe
```

Gambar 5.3 Git commit

Digunakan untuk menyimpan perubahan yang ada di staging area ke repository Git disertai dengan pesan commit sebagai keterangan perubahan.

5.4 GIT Remote

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git remote add origin https://github.com/muhammadazhriltirtha-debug/praktikum-apl.git
```

Gambar 5.4 Git remote

Digunakan untuk menghubungkan repository lokal dengan repository jarak jauh (misalnya di GitHub), sehingga kode dapat dibagikan atau disinkronkan.

5.5 GIT Push

```
PS D:\praktikum-apl\post-test\post-test-apl-1> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 19.95 KiB | 3.99 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/muhammadazhriltirtha-debug/praktikum-apl.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.5 Git push

Digunakan untuk mengirim (upload) commit dari repository lokal ke repository remote, sehingga perubahan dapat tersimpan secara online.